

**Universidade do Minho**  
Escola de Engenharia  
Licenciatura em Engenharia Informática

# **Computação Gráfica**

## **Trabalho Prático - Fase 4**

Grupo 15

|                       |                       |                   |                    |
|-----------------------|-----------------------|-------------------|--------------------|
| <b>Edgar Ferreira</b> | <b>Fernando Pires</b> | <b>Sara Silva</b> | <b>Zita Duarte</b> |
| a99890                | a77399                | a104608           | a104268            |

|                 |  |
|-----------------|--|
| Data da Receção |  |
| Responsável     |  |
| Avaliação       |  |
| Observações     |  |

# Computação Gráfica

Edgar Ferreira

a99890

Fernando Pires

a77399

Sara Silva

a104608

Zita Duarte

a104268

maio, 2025

# Índice

|                                     |           |
|-------------------------------------|-----------|
| <b>1. Introdução</b>                | <b>1</b>  |
| <b>2. Generator</b>                 | <b>2</b>  |
| 2.1. Ficheiros .3d                  | 2         |
| 2.2. Plano                          | 2         |
| 2.2.1. Normais                      | 2         |
| 2.2.2. Coordenadas de Textura       | 2         |
| 2.3. Caixa                          | 3         |
| 2.3.1. Normais                      | 3         |
| 2.3.2. Coordenadas de Textura       | 3         |
| 2.4. Esfera                         | 4         |
| 2.4.1. Normais                      | 4         |
| 2.4.2. Coordenadas de Textura       | 4         |
| 2.5. Cone                           | 5         |
| 2.5.1. Normais                      | 5         |
| 2.5.2. Coordenadas de Textura       | 5         |
| 2.6. Tórus                          | 5         |
| 2.6.1. Normais                      | 5         |
| 2.6.2. Coordenadas de Textura       | 6         |
| 2.7. Helix                          | 6         |
| 2.7.1. Normais                      | 6         |
| 2.7.2. Coordenadas de Textura       | 6         |
| 2.8. Patch                          | 7         |
| 2.8.1. Normais                      | 7         |
| 2.8.2. Coordenadas de Textura       | 7         |
| <b>3. Engine</b>                    | <b>8</b>  |
| 3.1. Material                       | 8         |
| 3.2. Lights                         | 8         |
| 3.3. Model                          | 9         |
| 3.3.1. prepareModel()               | 10        |
| 3.3.1.1. draw()                     | 10        |
| 3.3.1.2. drawNormals()              | 11        |
| 3.3.1.3. loadTexture()              | 11        |
| 3.3.1.4. initMaterial()             | 11        |
| 3.3.2. VBOs (Vertex Buffer Objects) | 11        |
| 3.3.3. IBOs (Index Buffer Objects)  | 12        |
| <b>4. Sistema Solar</b>             | <b>13</b> |
| <b>5. Testes</b>                    | <b>14</b> |
| <b>6. Conclusões</b>                | <b>15</b> |

## Lista de Figuras

|          |                                                                                     |    |
|----------|-------------------------------------------------------------------------------------|----|
| Figura 1 | Excerto exemplar de um ficheiro .3d, de acordo com a <b>nova estrutura</b> definida | 2  |
| Figura 2 | Excerto exemplar de um ficheiro .3d na <b>estrutura anterior</b>                    | 2  |
| Figura 3 | Cena do Sistema Solar com texturas, cores e luzes                                   | 13 |
| Figura 4 | Teste nº1                                                                           | 14 |
| Figura 5 | Teste nº2                                                                           | 14 |
| Figura 6 | Teste nº3                                                                           | 14 |
| Figura 7 | Teste nº4                                                                           | 14 |
| Figura 8 | Teste nº5                                                                           | 14 |
| Figura 9 | Teste nº6                                                                           | 14 |

# 1. Introdução

Este trabalho foi desenvolvido no âmbito da unidade curricular de Computação Gráfica (CG) no ano letivo 2024/2025.

A quarta fase do projeto teve como objetivo a introdução de novas funcionalidades tanto no **engine** como no **generator**. No *generator*, foi necessário adaptá-lo para que passasse a gerar vetores normais e coordenadas de textura para os vértices. Por sua vez, o *engine* foi modificado para receber e utilizar essas informações, além de suportar a adição de luzes, materiais e texturas.

## 2. Generator

### 2.1. Ficheiros .3d

Para permitir as alterações necessárias ao *generator*, referidas anteriormente, foi necessário modificar a estrutura dos ficheiros `.3d` utilizados até então. Os excertos apresentados abaixo ilustram a nova estrutura dos ficheiros `.3d`, em contraste com a versão anterior.

```
# 24
p -1 0 -1 0 1 0 0 0
p -1 0 0 0 1 0 0 0.5
p 0 0 0 0 1 0 0.5 0.5
p -1 0 -1 0 1 0 0 0
p 0 0 0 0 1 0 0.5 0.5
p 0 0 -1 0 1 0 0.5 0
```

Figura 1: Excerto exemplar de um ficheiro `.3d`, de acordo com a **nova estrutura** definida

```
-0.333333 0 -0.333333
-1 0 -1
-0.333333 0 -0.333333
-0.333333 0 -1
-0.333333 0 -1
-1 0 -0.333333
-1 0 -1
```

Figura 2: Excerto exemplar de um ficheiro `.3d` na **estrutura anterior**

A nova estrutura introduz uma linha inicial no formato `# <nr>`, que indica o número total de linhas do ficheiro, permitindo assim otimizar o processo de leitura. Adicionalmente, cada linha correspondente a um vértice passou a incluir, além da posição, também a normal e as coordenadas de textura, seguindo o formato:

```
p <posX posY posZ> <normX normY normZ> <textX textY>
```

onde **pos**, **norm** e **text** representam, respetivamente, as coordenadas de posição, normal e textura.

Esta nova estrutura assegura a inclusão de toda a informação necessária à aplicação de iluminação e texturas por parte do *engine*.

### 2.2. Plano

#### 2.2.1. Normais

No caso do Plano, todos os pontos partilham a mesma normal, uma vez que se encontram sobre a mesma superfície. Sabendo que o plano está contido no plano `xz`, deduz-se que a sua normal corresponde ao vetor  $(0, 1, 0)$ , ou seja, o vetor unitário orientado segundo o eixo dos `y`.

#### 2.2.2. Coordenadas de Textura

No caso do plano, a textura é aplicada de forma repetida ao longo da sua superfície, sendo ajustada a cada subdivisão. As coordenadas de textura são calculadas com base na posição atual da subdivisão dentro da grelha e no número total de divisões definido para o plano. Como os índices das subdivisões variam entre 0 e o número total de divisões, os valores resultantes das coordenadas de textura encontram-se sempre no intervalo  $[0, 1]$ , o que é compatível com a forma como as texturas são aplicadas no *engine*.

O código apresentado de seguida exemplifica este processo, onde, para cada subdivisão, são gerados dois triângulos com as respetivas posições, normais e coordenadas de textura:

```
float u1 = static_cast<float>(j) / divisions;
float v1 = static_cast<float>(i) / divisions;
float u2 = static_cast<float>(j + 1) / divisions;
float v2 = static_cast<float>(i + 1) / divisions;

// Triângulo 1
textures.push_back(Point2D(u1, v1));
textures.push_back(Point2D(u1, v2));
textures.push_back(Point2D(u2, v2));

// Triângulo 2
textures.push_back(Point2D(u1, v1));
textures.push_back(Point2D(u2, v2));
textures.push_back(Point2D(u2, v1));
```

## 2.3. Caixa

### 2.3.1. Normais

No caso da caixa, as normais dos vértices são determinadas unicamente pela face a que pertencem, sendo constantes em cada uma delas. Isto significa que todos os pontos de uma mesma face partilham a mesma normal, perpendicular ao respetivo plano. Assim, os vetores normais possíveis são:

- Topo: (0, 1, 0)
- Base: (0, -1, 0)
- Laterais:
  - Frente: (0, 0, 1)
  - Trás: (0, 0, -1)
  - Esquerda: (-1, 0, 0)
  - Direita: (1, 0, 0)

### 2.3.2. Coordenadas de Textura

Em cada face da caixa, a textura é aplicada de forma uniforme, sendo mapeada a cada subdivisão. Para tal, as coordenadas de textura são calculadas diretamente com base nos índices da grelha de subdivisão, sendo normalizadas pelo número total de divisões. Desta forma, os valores das coordenadas de textura variam entre 0 e 1, assegurando uma aplicação correta e contínua da textura sobre cada face.

O seguinte excerto de código ilustra este processo:

```

float u1 = static_cast<float>(j) / divisions;
float v1 = static_cast<float>(i) / divisions;
float u2 = static_cast<float>(j + 1) / divisions;
float v2 = static_cast<float>(i + 1) / divisions;

textures.push_back(Point2D(u1, v1));
textures.push_back(Point2D(u2, v1));
textures.push_back(Point2D(u1, v2));

textures.push_back(Point2D(u2, v2));
textures.push_back(Point2D(u1, v2));
textures.push_back(Point2D(u2, v1));

```

## 2.4. Esfera

### 2.4.1. Normais

Ao contrário do plano e da caixa, o cálculo das normais na esfera requer um processo adicional, uma vez que estas variam continuamente ao longo da superfície. Sabendo que a normal num ponto de uma esfera corresponde ao vetor que liga o centro da esfera ao próprio ponto, e considerando que o centro está fixo na origem (0, 0, 0), conclui-se que a normal de cada ponto é equivalente às suas próprias coordenadas. Desta forma, basta normalizar as coordenadas do ponto gerado na superfície da esfera para obter o vetor normal correspondente.

### 2.4.2. Coordenadas de Textura

A geração das coordenadas de textura para a esfera segue uma abordagem semelhante à utilizada no plano, mas adaptada à sua geometria esférica. Em vez de iterar sobre subdivisões cartesianas, o processo é feito com base nos ângulos *theta* (latitude) e *phi* (longitude), que definem os pontos sobre a superfície da esfera.

As coordenadas de textura são calculadas diretamente a partir desses ângulos, garantindo valores normalizados no intervalo [0, 1], através das seguintes expressões:

- $u = \phi / (2\pi)$  – mapeamento horizontal (longitude)
- $v = \theta / \pi$  – mapeamento vertical (latitude)

O seguinte excerto de código ilustra o cálculo destas coordenadas:

```

float u1 = phi1 / (2.0f * static_cast<float>(M_PI));
float u2 = phi2 / (2.0f * static_cast<float>(M_PI));
float v1 = theta1 / static_cast<float>(M_PI);
float v2 = theta2 / static_cast<float>(M_PI);

Point2D texture1(u1, v1);
Point2D texture2(u2, v1);
Point2D texture3(u1, v2);
Point2D texture4(u2, v2);

```

Estas coordenadas são depois associadas aos respetivos vértices da esfera, garantindo uma aplicação contínua e realista da textura sobre toda a superfície.



## 2.5. Cone

### 2.5.1. Normais

No cone, a base é um círculo situado no plano  $XZ$ . Para todos os pontos pertencentes à base, a normal é constante, apontando na direção negativa do eixo dos  $Y$ . Assim, todos esses pontos têm como vetor normal  $(0, -1, 0)$ .

Já a superfície lateral requer um cálculo mais cuidadoso, pois as normais variam de ponto para ponto, sendo perpendiculares à superfície inclinada do cone. Para obter estas normais, itera-se sobre os *slices* e *stacks*, utilizando um ângulo  $a$  e uma altura  $y$ . A normal num ponto da lateral pode ser calculada através do vetor  $(\sin(a), \text{raio\_base} / \text{altura}, \cos(a))$ , que representa a direção perpendicular à superfície do cone nesse ponto. Este vetor é posteriormente normalizado para garantir que é unitário.

### 2.5.2. Coordenadas de Textura

Além das normais, é também necessário mapear corretamente a textura à geometria do cone, tanto na base como na superfície lateral.

Para a base — um círculo no plano  $XZ$  — utiliza-se um mapeamento polar centrado na coordenada  $(0.5, 0.5)$  da textura, de forma a adaptar o círculo ao espaço quadrado da imagem. A conversão é feita com as seguintes fórmulas:

```
Point2D(0.5f + 0.5f * sin(theta), 0.5f + 0.5f * cos(theta));
```

Já na superfície lateral, onde as normais são calculadas ponto a ponto para refletir a inclinação da face, é necessário aplicar um mapeamento cilíndrico para representar a textura em 2D. Aqui,  $u$  representa a componente angular (relacionada com o ângulo  $\theta$  da *slice*), enquanto  $v$  representa a componente vertical (relacionada com a altura da *stack*). Ambos os valores são normalizados para o intervalo  $[0, 1]$ :

```
float u1 = static_cast<float>(i) / slices;  
float u2 = static_cast<float>(i + 1) / slices;  
float v1 = static_cast<float>(j) / stacks;  
float v2 = static_cast<float>(j + 1) / stacks;
```

## 2.6. Tórus

### 2.6.1. Normais

No *torus*, o cálculo das normais é feito de forma diferenciada em relação a sólidos como a esfera ou o cone. Neste caso, cada ponto da superfície pertence a um “tubo circular” que se desenvolve ao longo de um círculo principal.

Para cada ponto da superfície, a normal é obtida a partir do vetor que liga o centro da secção circular do tubo (localizado sobre o círculo principal do *torus*) até o ponto em questão. Como o *torus* está centrado na origem e o círculo principal está no plano  $XZ$ , o centro da secção circular, para um ângulo  $\theta$ , está em:

```
(majorRadius * cos(theta), 0, majorRadius * sin(theta))
```

Dado o ponto  $P$  da superfície, a normal resulta do vetor  $P - C$ , onde  $C$  é o centro da secção circular. Este vetor é depois normalizado para garantir comprimento unitário.

### 2.6.2. Coordenadas de Textura

A aplicação da textura no *torus* é feita através de mapeamento baseado nas duas dimensões angulares da sua superfície: *theta* (ângulo ao longo do círculo principal) e *phi* (ângulo da secção circular).

As coordenadas de textura são atribuídas a cada ponto com base na sua posição relativa ao número total de *rings* e *sides* (divisões do círculo principal e do tubo, respetivamente). Os valores de *u* e *v* são normalizados para o intervalo [0, 1]:

```
float u1 = static_cast<float>(j) / static_cast<float>(sides);
float u2 = static_cast<float>(j + 1) / static_cast<float>(sides);
float v1 = static_cast<float>(i) / static_cast<float>(rings);
float v2 = static_cast<float>(i + 1) / static_cast<float>(rings);
```

## 2.7. Helix

### 2.7.1. Normais

No caso da hélice, a geometria é composta por várias superfícies triangulares dispostas ao longo de uma curva helicoidal. Como não existe uma fórmula direta para obter a normal em cada ponto da superfície, estas são calculadas com base nos triângulos formados em cada iteração.

Para cada triângulo, a normal é calculada com recurso à função `computeNormal`, que implementa o seguinte procedimento:

```
void computeNormal(float* a, float* b, float* c, float* res) {
    float u[3], v[3];
    for (int i = 0; i < 3; i++) {
        u[i] = b[i] - a[i];
        v[i] = c[i] - a[i];
    }
    cross(u, v, res); // Produto vetorial
    normalize(res);   // Normalização do vetor
}
```

Este método segue três passos:

1. Calcula dois vetores *u* e *v* a partir dos pontos *a*, *b* e *c* do triângulo;
2. Obtém o vetor perpendicular ao plano definido por esses pontos através do produto vetorial `cross(u, v)`;
3. Normaliza o vetor resultante, garantindo que a normal tem comprimento unitário.

### 2.7.2. Coordenadas de Textura

A aplicação da textura na hélice é realizada com base em duas componentes principais:

1. o ângulo da posição atual ao longo da espiral (circunferência);
2. a altura vertical do ponto na estrutura.

Para garantir que a textura é corretamente mapeada à geometria helicoidal, as coordenadas de textura são normalizadas no intervalo [0, 1] com as seguintes fórmulas:

- $u = \text{angle} / (2 * \pi)$
- $v = (y + \text{height} / 2) / \text{height}$

O valor *u* representa a componente horizontal da textura, proporcional ao avanço angular da hélice.

O valor *v* representa a componente vertical, determinada pela altura *y* do ponto em relação ao centro da hélice (altura deslocada para cima para ficar no intervalo [0, *height*]).

No código, esta lógica está espelhada nas chamadas a `textures.push_back(...)`, onde as coordenadas são calculadas dinamicamente para cada vértice dos triângulos gerados:

```
textures.push_back(Point2D(angle / (2 * M_PI), (y + height / 2) / height));
```

## 2.8. Patch

### 2.8.1. Normais

No caso dos *patches* de Bézier, a superfície é definida por um conjunto de 16 pontos de controlo dispostos numa grelha 4x4. Esta superfície é avaliada através da aplicação da fórmula do Bézier bicúbico para dois parâmetros `u` e `v`, variando entre 0 e 1, que definem a posição relativa sobre o *patch*.

Como a superfície é curva e suave, as normais variam continuamente ao longo da mesma. Para cada ponto, a normal é calculada utilizando a derivada da função de Bézier em relação aos parâmetros `u` e `v`, ou seja, com base nos vetores tangentes  $\partial P / \partial u$  e  $\partial P / \partial v$ . Estes dois vetores são obtidos na função `bezierNormal`, onde:

1. Se aplicam as derivadas das funções de Bernstein (através de `binomialDeriv`);
2. Se calculam os vetores tangentes nas direções `u` e `v` (`du` e `dv`);
3. Se obtém a normal como o produto vetorial entre `dv` e `du`;
4. O vetor resultante é normalizado, garantindo que tem comprimento unitário.

### 2.8.2. Coordenadas de Textura

A aplicação de texturas a superfícies definidas por *patches* de Bézier é feita de forma direta e proporcional, utilizando os próprios parâmetros `u` e `v` que são usados para gerar os pontos da superfície.

Cada coordenada `(u, v)` representa uma posição relativa dentro do *patch*, variando entre 0 e 1. Como tal, estas mesmas coordenadas são reutilizadas como coordenadas de textura, resultando num mapeamento natural e uniforme da imagem sobre a superfície.

```
Point2D texCoord(u, v);
```

No código, são atribuídas coordenadas de textura a cada vértice de triângulo da seguinte forma:

```
textures.push_back(Point2D(u, v));
textures.push_back(Point2D(u, v1));
textures.push_back(Point2D(u1, v));
```

## 3. Engine

### 3.1. Material

De forma a permitir o uso de materiais nos modelos, foi criada a seguinte estrutura de dados:

```
struct Material {  
    float ambient[4] = {0.2f, 0.2f, 0.2f, 1.0f};  
    float diffuse[4] = {0.8f, 0.8f, 0.8f, 1.0f};  
    float specular[4] = {0.0f, 0.0f, 0.0f, 1.0f};  
    float emission[4] = {0.0f, 0.0f, 0.0f, 1.0f};  
    float shininess = 0.0f;  
};
```

Nesta estrutura são guardadas as principais componentes que definem o comportamento do material perante a luz: a componente ambiente ( `ambient` ), a componente difusa ( `diffuse` ), a componente especular ( `specular` ), a componente emissiva ( `emission` ), e o valor de brilho ( `shininess` ).

Com estes valores definidos, podemos configurar o material para ser usado durante o processo de renderização com `OpenGL`. Para tal, foi implementado o método `initMaterial()`, que aplica os valores da estrutura `Material` através das funções `glMaterialfv` e `glMaterialf`:

```
void Model::initMaterial() {  
    glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, this->material.ambient);  
    glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, this->material.diffuse);  
    glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, this->material.specular);  
    glMaterialfv(GL_FRONT_AND_BACK, GL_EMISSION, this->material.emission);  
    glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, this->material.shininess);  
}
```

### 3.2. Lights

Tal como no caso dos materiais, foi criada uma estrutura própria para representar as luzes utilizadas na cena. A estrutura `Light` contém a informação necessária para definir três tipos de luzes diferentes: direcional, ponto e *spot*. A definição da estrutura é a seguinte:

```
enum LType { D, P, S, E };  
  
struct Light {  
    LType type = E;  
    float pos[4] = { 0, 0, 0, 1 };  
    float dir[4] = { 0, 0, 0, 0 };  
    float cut = 180;  
};
```

Esta estrutura guarda:

1. o tipo da luz ( `type` ) — Direcional (D), Ponto (P), Spot (S) ou Nenhuma (E);
2. a posição da luz ( `pos` ) — usada em luzes de ponto e *spot*;
3. a direção da luz ( `dir` ) — usada em luzes direcionais e *spot*;
4. o cutoff ( `cut` ) — ângulo de abertura do feixe, utilizado apenas nas luzes *spot*.

Para ativar e configurar as luzes no contexto do `OpenGL`, foi implementada a função `initLights()`, que ativa o sistema de iluminação global e associa até 8 luzes (de `GL_LIGHT0` a `GL_LIGHT7`) com valores padrão para os componentes `diffuse` e `specular`:

```
bool Configuration::initLights() {
    if (this->lights.size() > 0) {
        glEnable(GL_RESCALE_NORMAL);
        float ambient[4] = { 1.0f, 1.0f, 1.0f, 1.0f };
        glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ambient);
        glEnable(GL_LIGHTING);
        for (size_t i = 0; i < this->lights.size() && i < 8; i++) {
            float white[4] = { 1.0, 1.0, 1.0, 1.0 };
            glEnable(GL_LIGHT0 + i);
            glLightfv(GL_LIGHT0 + i, GL_DIFFUSE, white);
            glLightfv(GL_LIGHT0 + i, GL_SPECULAR, white);
        }
        return true;
    }
    return false;
}
```

Por fim, a função `drawLights()` trata da configuração concreta de cada luz, com base no seu tipo:

```
void Configuration::drawLights() {
    for (size_t i = 0; i < lights.size() && i < 8; i++) {
        const Light& light = lights[i];
        if (light.type == LType::D) {
            glLightfv(GL_LIGHT0 + i, GL_POSITION, light.dir);
        }
        else if (light.type == LType::P) {
            glLightfv(GL_LIGHT0 + i, GL_POSITION, light.pos);
        }
        else if (light.type == LType::S) {
            glLightfv(GL_LIGHT0 + i, GL_POSITION, light.pos);
            glLightfv(GL_LIGHT0 + i, GL_SPOT_DIRECTION, light.dir);
            glLightf(GL_LIGHT0 + i, GL_SPOT_CUTOFF, light.cut);
        }
    }
}
```

Esta função permite renderizar corretamente todos os tipos de luzes definidos na cena. Luzes direcionais são configuradas apenas com a direção; luzes de ponto usam apenas a posição; e luzes *spot* utilizam a posição, direção e o *cutoff* para definir o feixe de luz projetado.

### 3.3. Model

Durante esta fase do projeto foi necessário expandir e refinar a classe `Model` de forma a suportar as funcionalidades exigidas, como texturas, materiais, normais e armazenamento eficiente dos vértices. Esta classe representa um modelo 3D individual e contém toda a informação necessária para o desenhar corretamente com `OpenGL`.

A estrutura da classe é a seguinte:

```
class Model {
public:
    std::string textureFile = "";
    Material material = Material();

    Model();
    // Construtor atualizado
    Model(const std::vector<Point>& vertices,
          const std::vector<Point>& normals,
          const std::vector<Point2D>& textures,
          const std::string& textureFile = "",
          const Material& mat = Material());

    void draw(bool norm, bool light);
    void prepareModel();
    void initMaterial();
    void loadTexture();
    void drawNormals(bool light);

private:
    std::vector<Point2D> textures;
    std::vector<Point> vertices;
    std::vector<unsigned int> indices;
    std::vector<Point> normalsData;
    GLuint vbo = 0, ebo = 0, normals = 0, textureId = 0, textureVBO = 0;;
};
```

Foram adicionados os campos `material`, `textureFile` e `textureId`, bem como os identificadores de buffer para os vértices (`vbo`), normais (`normals`), texturas (`textureVBO`) e índices (`ebo`). O modelo é construído a partir dos seus pontos, normais e coordenadas de textura, podendo opcionalmente carregar uma textura e aplicar materiais definidos com `glMaterial`.

De forma a otimizar a arquitetura, também optamos por remover os **VAOs** implementados na última fase, visto que não traziam benefícios no nosso contexto. Sendo assim, mantivemos apenas os buffers essenciais (tal como ensinado nas aulas), simplificando a estrutura e tornando o código mais direto e eficiente para o nosso caso de uso específico.

### 3.3.1. prepareModel()

A função `prepareModel()` tem como objetivo eliminar vértices duplicados com base na posição, normal e coordenada de textura, criando assim um índice otimizado para envio à GPU:

```
std::string key = pos + "|" + normal + "|" + texture;
```

Com os dados únicos extraídos, são criados os **VB0s** (*Vertex Buffer Objects*) e **IB0** (*Index Buffer Object*) usando as funções `glGenBuffers`, `glBindBuffer` e `glBufferData`.

Caso o modelo tenha uma textura associada, a função chama automaticamente `loadTexture()` para carregar e aplicar a textura ao `textureId`.

#### 3.3.1.1. draw()

A função `draw()` é responsável por desenhar o modelo. É aqui que os *buffers* são ativados e os dados são desenhados com `glDrawElements`:

```
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glVertexPointer(...);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);
glDrawElements(GL_TRIANGLES, ...);
```

Se `light == true`, é ativado o sistema de iluminação com `glEnable(GL_LIGHTING)` e aplicado o material através da função `initMaterial()`.

Caso exista uma textura associada, é ativada com `glBindTexture`, e as coordenadas de textura são ligadas com `glTexCoordPointer`.

Após o desenho, os *client states* e *buffers* são desativados para garantir que não interferem com outros objetos da cena.

#### 3.3.1.2. drawNormals()

Esta função é útil para depuração visual e permite desenhar as normais do modelo em tempo real. São desenhadas linhas vermelhas que partem de cada vértice e seguem a direção da sua normal:

```
glBegin(GL_LINES);
glVertex3f(v.x, v.y, v.z);
glVertex3f(v.x + n.x * scale, v.y + n.y * scale, v.z + n.z * scale);
glEnd();
```

Se a iluminação estiver ativa, é temporariamente desativada para garantir que as linhas se destacam na cena.

#### 3.3.1.3. loadTexture()

A função `loadTexture()` utiliza a biblioteca `DevIL` para carregar texturas a partir do diretório `textures/`. O ficheiro é lido, convertido para o formato RGBA e enviado para a GPU com `glTexImage2D`. É ainda ativado o Mipmapping com `glGenerateMipmap`, e definidos os parâmetros de textura:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
```

Note-se que, embora o Mipmapping esteja ativado, o filtro usado neste projeto é `GL_LINEAR`. Caso se pretenda ativar realmente a visualização do Mipmapping, com `GL_LINEAR_MIPMAP_LINEAR`, basta descomentar a respetiva linha ou trocar os respetivos parâmetros.

#### 3.3.1.4. initMaterial()

Esta função aplica o material do modelo às faces com `GL_FRONT_AND_BACK`, utilizando as componentes definidas na `struct Material`. Os valores de *ambient*, *diffuse*, *specular*, *emission* e *shininess* são carregados para o pipeline de iluminação do `OpenGL`, garantindo uma renderização visualmente correta quando `GL_LIGHTING` está ativo.

#### 3.3.2. VBOs (Vertex Buffer Objects)

A construção dos VBOs foi realizada na função `prepareModel()`. Nesta função, ao invés de considerarmos apenas a posição dos vértices, foi necessário considerar também as respetivas normais e coordenadas de textura.

Desta forma, para cada vértice, é criada uma chave única composta pela tripla (posição, normal, coordenada de textura). Esta abordagem garante que apenas vértices únicos são armazenados nos *buffers*, evitando redundâncias e permitindo uma otimização da memória enviada para a GPU.

Após identificados os vértices únicos, os respectivos *buffers* são gerados com:

```
glGenBuffers(...);  
glBindBuffer(...);  
glBufferData(...);
```

São gerados três VBOs distintos: um para os vértices (vbo), um para as normais (normals) e um para as coordenadas de textura (textureVBO).

### 3.3.3. IBOs (Index Buffer Objects)

Em paralelo à construção dos VBOs, também foi gerado um IBO, o qual contém os índices dos vértices únicos para formar os triângulos necessários ao desenho do modelo.

Este IBO é armazenado na variável `ebo` e preenchido também na função `prepareModel()`, após a identificação dos vértices únicos.

O envio do *buffer* para a GPU é feito da seguinte forma:

```
glGenBuffers(1, &ebo);  
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);  
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(unsigned int),  
indices.data(), GL_STATIC_DRAW);
```

Com esta abordagem, conseguimos desenhar os modelos de forma eficiente, utilizando `glDrawElements` e evitando redundâncias nos dados.



## 4. Sistema Solar

Como forma de melhor testar as novas funcionalidades incluídas nesta fase, melhoramos a cena relativa ao Sistema Solar adicionando texturas, materiais e luzes.

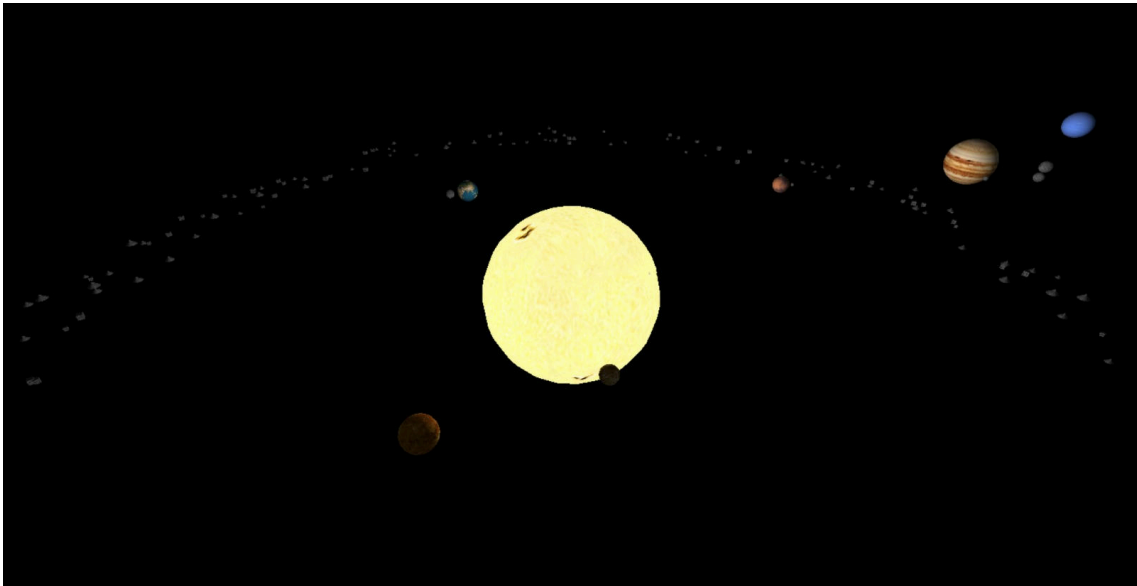


Figura 3: Cena do Sistema Solar com texturas, cores e luzes

Para alcançar este resultado, atualizamos o gerador XML `generator solar` para incluir os novos atributos e nodos necessários para utilização das novas componentes.

## 5. Testes

De modo a testar as diversas funcionalidades implementadas nesta fase, foram realizados um conjunto de testes que se apresentam de seguida.

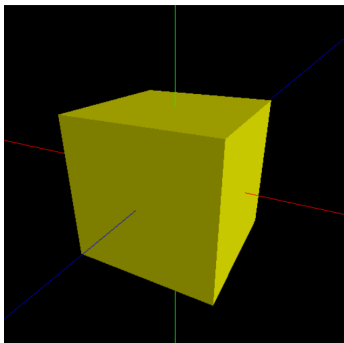


Figura 4: Teste nº1

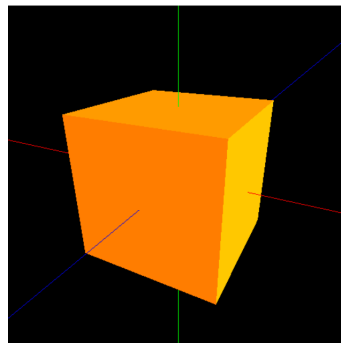


Figura 5: Teste nº2

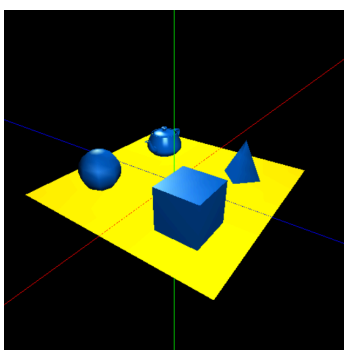


Figura 6: Teste nº3

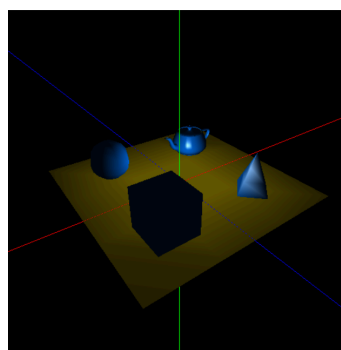


Figura 7: Teste nº4

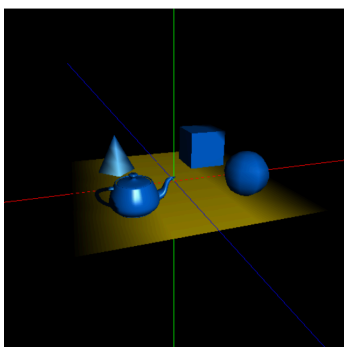


Figura 8: Teste nº5

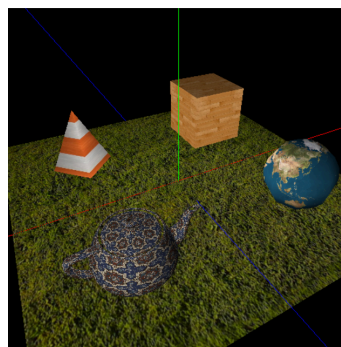


Figura 9: Teste nº6

Além dos testes disponibilizados, implementamos testes adicionais para verificar o conjunto de componentes desta e últimas fases.

Todos os testes executados apresentaram resultados conforme o esperado.

## 6. Conclusões

Consideramos que os resultados obtidos nesta fase corresponderam às expectativas iniciais, ainda que algumas otimizações, nomeadamente ao nível da gestão de memória ou de certos cálculos, não tenham sido plenamente alcançadas.

Apesar dessas limitações, todo o processo de desenvolvimento revelou-se extremamente enriquecedor. Esta fase permitiu-nos consolidar e aplicar os conhecimentos adquiridos ao longo da unidade curricular de Computação Gráfica, ao mesmo tempo que nos proporcionou uma experiência prática próxima da realidade de desenvolvimento de motores gráficos e manipulação de modelos 3D num contexto académico.